État de l'art : bibliothèques JS de visualisation de graphes

Oct 20, 2026 · @ALLEY

Recommandation : **sigma.js (avec sa bibliothèque compagnon graphology)** pour le PoC et la suite, avec **Cytoscape.js** comme solution de repli si l'on privilégie un style très riche sur de petits graphes. Le reste du document explique pourquoi, puis décrit le PoC.

1. Contexte et besoin

Poppy calcule le PageRank d'un graphe découpé entre plusieurs data centers qui ne se font pas confiance, en gardant les calculs chiffrés. On a maintenant besoin de **voir** ce graphe et ses résultats dans un navigateur web, pour comprendre, expliquer et déboguer.

Concrètement, la visualisation doit montrer :

- les nœuds (ici des étudiants) et les arêtes (qui pointe vers qui) ;
- **à quel data center** appartient chaque nœud (une couleur par data center) ;
- les **arêtes inter-data centers**, celles qui obligent Poppy à échanger des chiffrés (ce sont elles qui coûtent cher) ;
- le **score PageRank** de chaque nœud (taille du nœud), et idéalement son évolution itération après itération ;
- les nœuds particuliers de l'article : **pendants** (sans arête sortante, « Dang. ») et **constants** (sans arête entrante, « Const. »).

Le graphe Students est petit (185 nœuds, 311 arêtes, tableau 2 de l'article). Mais l'article utilise aussi Slashdot (77 360 nœuds, 905 468 arêtes). Si l'outil doit un jour afficher ces gros graphes, **la capacité à passer à l'échelle devient le critère qui départage les bibliothèques**.

2. Les notions à connaître avant de comparer

Une bibliothèque de visualisation de graphes fait trois choses : **stocker** le graphe, **placer** les nœuds à l'écran, puis les **dessiner**. Chaque bibliothèque fait des choix différents sur ces trois points.

Comment on dessine dans un navigateur. Il existe trois techniques, et c'est elle qui fixe la taille maximale du graphe affichable.

Technique	Image simple	Taille confortable (ordre de grandeur)	Avantage	Inconvénient
SVG	Chaque nœud est un petit objet de la page, comme un bouton	Quelques centaines à ~2 000 éléments	Très facile à styler et à cliquer	Devient lent vite
Canvas 2D	Le navigateur peint une image, pixel par pixel, avec le processeur	Quelques milliers à ~10 000 éléments	Bon compromis	Il faut gérer soi-même les clics
WebGL	On confie le dessin à la carte graphique (GPU), comme un jeu vidéo	Dizaines de milliers à millions d'éléments	Très rapide	Formes et styles plus limités, plus technique à personnaliser

Ces ordres de grandeur sont approximatifs : ils dépendent de la machine et du nombre d'arêtes.

Le layout (la mise en page). Un graphe n'a pas de position naturelle : il faut calculer où placer chaque nœud. L'algorithme le plus courant est le *force-directed* : on imagine que les arêtes sont des ressorts et que les nœuds se repoussent comme des aimants. ForceAtlas2, créé pour le logiciel Gephi, en est une version très utilisée.

Le modèle de graphe. C'est la structure de données en mémoire (liste des nœuds, des arêtes, de leurs attributs). Certaines bibliothèques y ajoutent des algorithmes tout faits, dont **PageRank**, ce qui nous intéresse directement pour vérifier les résultats de Poppy.

3. Panorama des bibliothèques existantes

On retient huit bibliothèques libres et utilisables directement dans une page web, plus quelques solutions commerciales citées pour mémoire. Versions et licences relevées sur le registre npm le 20/10/2026.

Bibliothèque	Rendu	Version / licence	En une phrase	Point fort	Point faible
sigma.js + graphology	WebGL	3.0.2 stable, 4.0 sortie le 08/10/2026 / MIT	Le « Gephi du navigateur », fait par le Médialab de Sciences Po	Grands graphes fluides ; graphology fournit PageRank, ForceAtlas2, Louvain	Personnalisation des formes plus technique (programmes WebGL)
Cytoscape.js	Canvas (WebGL expérimental)	3.34 / MIT	Bibliothèque académique très complète, issue de la bio-informatique	Style façon CSS, nombreux layouts, PageRank intégré, nœuds « parents » pour regrouper	Ralentit au-delà de quelques milliers d'éléments
vis-network	Canvas	10.1 / Apache-2.0 ou MIT	La plus simple pour démarrer	Physique et interactions prêtes à l'emploi	Lent sur les gros graphes, pas d'algorithmes de graphe
D3.js (d3-force)	SVG ou Canvas	7.9 / ISC	Boîte à outils générale de dataviz	Liberté totale, énorme communauté	Tout est à coder, courbe d'apprentissage raide
AntV G6	Canvas, SVG ou WebGL	5.1 / MIT	Framework complet du groupe Ant (Alibaba)	Très riche (layouts, plugins, animations)	API lourde, une partie de la doc et de la communauté en chinois
cosmos.gl	WebGL (layout sur GPU)	3.5 / MIT	Moteur qui calcule le layout sur la carte graphique	Des centaines de milliers de nœuds	Peu de styles et d'interactions, peu d'algorithmes
force-graph / 3d-force-graph	Canvas ou WebGL (3D)	1.52 / MIT	Petit composant prêt à l'emploi, basé sur d3-force	Très peu de code, version 3D	Peu d'algorithmes, maintenu par une seule personne
Reagraph	WebGL	4.32 / Apache-2.0	Composant React	Joli par défaut, 2D et 3D	Impose React

Solutions commerciales (exclues pour la licence). Ogma de Linkurious, KeyLines/ReGraph de Cambridge Intelligence et yFiles for HTML sont très performantes mais payantes. Elles sont peu adaptées à un projet de recherche open source.

Hors sujet mais souvent cités. Graphviz (via viz.js) et ELK produisent des schémas hiérarchiques statiques. Ils conviennent à des diagrammes, pas à l'exploration interactive d'un réseau social.

4. Grille d'évaluation

sigma.js arrive en tête avec 4,45/5, devant Cytoscape.js (4,10) et G6 (3,85). Chaque bibliothèque est notée de 1 (mauvais) à 5 (excellent) sur sept critères, pondérés selon le besoin de Poppy.

Les critères et leur poids :

1. **Échelle (25 %)** : fluidité jusqu'à Slashdot (77 000 nœuds, 900 000 arêtes).
2. **Algorithmes (15 %)** : PageRank et layouts disponibles, pour comparer avec les scores de Poppy.
3. **Personnalisation (15 %)** : couleur par data center, arêtes inter-DC mises en avant, taille selon le score.
4. **Facilité (15 %)** : temps pour un débutant avant un premier résultat.
5. **Documentation et communauté (10 %)** : exemples, réponses en ligne.
6. **Maintenance (10 %)** : projet actif, publications récentes.
7. **Licence et indépendance (10 %)** : licence libre, pas de framework imposé (React, etc.).

Bibliothèque	Échelle	Algorithmes	Personnalisation	Facilité	Doc	Maintenance	Licence	**Total /5**
sigma.js + graphology	5	5	3	4	4	5	5	**4,45**
Cytoscape.js	2	5	5	4	5	5	5	**4,10**
AntV G6	4	4	5	2	3	4	5	**3,85**
cosmos.gl	5	1	2	3	2	4	5	**3,25**
vis-network	2	1	4	5	4	3	5	**3,20**
force-graph	3	1	3	5	3	3	5	**3,20**
D3.js	2	1	5	2	5	4	5	**3,10**
Reagraph	4	2	3	3	3	4	2	**3,10**

Les notes sont mon appréciation à partir de la documentation et de l'usage courant de ces outils, pas d'un banc d'essai. Pour un rapport, il serait bon de les confirmer par une mesure simple : afficher le même graphe (par exemple WikiVote, 7 115 nœuds) avec les trois premières et noter les images par seconde.

Si l'on retire le critère d'échelle (on n'afficherait jamais que Students), Cytoscape.js passe devant (4,8 contre 4,3). Le choix dépend donc surtout de cette question à poser à l'encadreur.

5. Recommandation : sigma.js + graphology

sigma.js est le seul outil qui couvre à la fois le petit graphe Students et les gros graphes de l'article, tout en fournissant PageRank. Quatre raisons concrètes :

1. **Il tient la charge.** Le rendu WebGL reste fluide sur des dizaines de milliers de nœuds, donc on ne devra pas changer d'outil en passant de Students à WikiVote ou Slashdot.
2. **PageRank est déjà écrit.** graphology propose `pagerank`, ForceAtlas2 et Louvain. On peut donc afficher le PageRank « en clair » et le comparer aux scores déchiffrés de Poppy pour vérifier la précision.
3. **La séparation est claire.** graphology gère les données, sigma ne fait que dessiner. C'est plus facile à comprendre pour un débutant et à brancher plus tard sur les sorties de Poppy.
4. **Le projet est sérieux et libre.** Licence MIT, développé par des chercheurs français (Médialab Sciences Po et OuestWare), avec une version 4 sortie en octobre 2026.

Précautions. Pour le PoC, on reste sur la version 3.0.2, stable et bien documentée, plutôt que la 4.0 sortie il y a douze jours. Les formes de nœuds se limitent à des cercles par défaut, ce qui suffit ici.

Plan B : Cytoscape.js, si l'encadreur confirme qu'on n'affichera que de petits graphes. Son atout pour Poppy : les « nœuds composés » permettent de dessiner chaque data center comme une boîte contenant ses nœuds.

6. Conception du PoC sur le graphe Students

Un premier PoC fonctionne déjà : PoppyViz · graphe Students, une seule page HTML avec sigma.js 3.0.2 et graphology 0.26. Il tourne sur un **graphe d'exemple synthétique**, car les vraies données Students sont sur le GitLab Inria, inaccessible depuis mon environnement.

Le graphe d'exemple reprend les caractéristiques de Students données dans le tableau 2 de l'article : 185 nœuds, 311 arêtes orientées, 22 nœuds pendants, 17 nœuds constants, degré entrant maximal 6. Ses nœuds sont répartis entre 3 data centers (DC1, DC2, DC3). Les liens et la répartition sont inventés.

Objectif du PoC : montrer qu'on peut, dans un navigateur, voir le graphe tel que Poppy le découpe, et suivre le calcul du PageRank.

Ce que fait la page :

- couleur d'un nœud = son data center ; taille = son score PageRank ;
- arêtes inter-DC en rouge : ce sont celles qui obligent Poppy à envoyer des chiffrés ;
- curseur et bouton Lecture pour rejouer le PageRank itération par itération, avec la variation entre deux itérations ;
- **point de vue d'un data center** : on ne voit que ses nœuds et les nœuds distants qui lui sont reliés, ce qui illustre le fait qu'aucun participant ne voit tout le graphe ;
- mise en évidence des arêtes inter-DC, des nœuds pendants et des nœuds constants ;
- survol ou clic d'un nœud : voisins, score, degrés entrant et sortant, nombre d'arêtes inter-DC ;
- top 10 des scores ;
- chargement d'un autre fichier, pour brancher le vrai graphe Students dès qu'on l'aura.

Comment c'est construit, en quatre étapes :

1. **Lecture du fichier.** Un fichier texte avec une section `[noeuds]` (« id data_center ») et une section `[aretes]` (« source cible ») devient un graphe graphology.
2. **Calcul du PageRank en clair.** Une boucle de 30 lignes applique la formule PR(v) = (1 − d)/N + d × Σ PR(u)/sortant(u), avec d = 0,85, et garde le résultat de chaque itération. J'ai vérifié qu'elle donne les mêmes scores que la fonction `pagerank` de graphology.
3. **Disposition.** Chaque data center part d'un secteur du cercle, puis ForceAtlas2 (400 itérations) place les nœuds.
4. **Dessin.** sigma.js affiche le tout ; deux petites fonctions (« reducers ») décident de la couleur, de la taille et de la visibilité selon les réglages choisis.

La page HTML (poppyviz-students.html) et les données d'exemple (students_exemple.txt) sont aussi dans les fichiers du projet, dossier poc-visualisation. Le code est commenté en français, section par section.

Suites possibles, par ordre d'intérêt :

1. Charger le vrai graphe Students et sa vraie répartition entre data centers.
2. Charger les scores **déchiffrés** produits par Poppy et colorer chaque nœud selon l'écart avec le PageRank en clair, pour visualiser l'erreur due à CKKS.
3. Afficher l'élagage PINC : griser les nœuds retirés (76 sur Students d'après l'article).
4. Tester la même page sur WikiVote (7 115 nœuds) pour mesurer la fluidité et confirmer le choix de sigma.js.

7. Questions pour l'encadreur

1. Faudra-t-il afficher les gros graphes de l'article (WikiVote, Slashdot), ou seulement Students ? C'est ce qui décide entre sigma.js et Cytoscape.js.

2. Dans quel format sont le graphe Students et sa répartition entre data centers dans le dépôt ? Combien de data centers ?
3. Que doit montrer la visualisation : le graphe en clair (outil de démonstration), ou seulement ce que voit un data center (outil respectueux de la confidentialité) ?
4. Peut-on récupérer les scores déchiffrés de Poppy, itération par itération, pour les comparer au PageRank en clair ?
5. Y a-t-il une contrainte technique (React, intégration dans une application existante, usage hors ligne) ?
6. Faut-il visualiser l'élagage PINC ou le découpage en slots SIMD des chiffrés ?

Sources

Versions et licences relevées sur le registre npm le 20/10/2026. Caractéristiques de Students : tableau 2 et tableau des résultats d'élagage de l'article Poppy (PoPETs 2026).

- sigma.js et graphology
- Cytoscape.js
- vis-network
- D3.js
- AntV G6
- cosmos.gl
- force-graph
- Reagraph
- Ogma (Linkurious)